Debugging and profiling tools

Amitabha Sanyal

April 2026

Introduction: Debugging and Profiling Tools

Debugging and profiling tools are essential for identifying logic errors, memory issues, and performance bottlenecks in software.

- **gdb**: Interactive debugger for stepping through code, inspecting state, and diagnosing crashes.
- **gprof**: Function-level profiler that generates execution-time statistics and call graphs from sampling.
- **perf**: Linux performance analyzer leveraging hardware counters for fine-grained profiling and system tracing.
- **valgrind**: Instrumentation framework (e.g. Memcheck) for detecting memory leaks, invalid accesses, and threading errors.

The GNU Debugger (GDB)

Outline

- The GDB model
  - The debugger process and the debugged process
- Basic usage
  - Starting GDB
  - Setting breakpoints
  - Stepping over the program: next, step, finish
  - Examining data
  - Watchpoints
  - Examining the call stack
- Advanced gdb
  - Working with core dumps
  - Writing pretty-printers using the GDB Python API.

Compiling the program for debugging

- Assume that the program being debugged is **buggyprog.cpp**
  - Compile the program as **$ g++ -g buggyprog.cpp -o buggyprog**
  - **-g** provides GDB extra debugging information to connect the executable back to the source program.
    - What are the 10 source statements around the current breakpoint?
    - Which machine instructions correspond to:
      ```
      seriesValue += xpow/ComputeFactorial(k);
      ```
    - What is the type of the variable **seriesValue**?
    - What is the memory address where **seriesValue** is stored?
- The **-g** flag adds this information to the executable.
  - **readelf --debug-dump=line** gives line number → machine code map
  - **readelf --debug-dump=info** gives types, location (grep for the variable)

Starting the Debugger

- Start the debugger as:
  - `$ gdb buggyprog`
  - `$ gdb --args buggyprog` *args...* (arguments from the command line)
  - `$ gdb buggyprog` *coredump* (more about this later)

- Run the debugger as:
  - `$ (gdb) run`
  - `$ (gdb) run` *arg1 arg2*

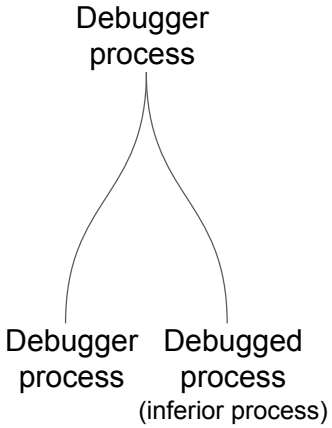
    And the program runs to completion (or abortion) with bugs and everything. Not very useful.

The GDB model

Debugger Process (gdb)

Reads user input, and executes it:

- Shows info about debugger state - breakpts, watchpts, convenience vars, macros)
- Shows information about the debugged process (program vars, PC).
- Enables breakpt setting
- Passes control to debugged process.



Debugged Process

- Executes till breakpoint or end.
- Forced to give up control to debugger.

Starting the Debugger: What happens internally

1. **GDB:** user types **`break main`** and then **`run`** (alternately **`start`**)
2. **GDB:** **`fork()`** → **`waitpid()`**
3. **Child:** **`ptrace(PTRACE_TRACEME)`** → **`execve("./buggyprog")`**
4. **Kernel:** stops child at **`execve`**, wakes **GDB** via **`waitpid()`.**
5. **GDB (Wakes):** **`set_breakpoint(main);`** hands control back to child
6. **Child (Runs):** Hits **`0xCC;`** Hardware triggers exception. Call this program point **`p`**.
7. **Kernel:** Halts child through **`SIGTRAP,`** wakes **GDB**.
8. **GDB (Wakes):** User debugs and configures additional breakpoints at various program points.
   a. For each additional program point **`p'`** do: **`set_breakpoint(p');`**
   b. **`execute_instr_at(p);`**
   c. **`ptrace(PTRACE_CONT); waitpid()`**

`execute_instr_at(p):` Restore original instruction at **`p`**; Execute this instruction; Write **`0xCC`** back to **`p`**.

`set_breakpoint(p):` Read and save original instruction at **`p`**; Write **`0xCC`**.

Setting Breakpoints

- Setting breakpoints
  - at line *lineno* of the current file
    (gdb) break *lineno*
  - Other variations
    (gdb) break *function*
    (gdb) break *filename:linenum*
    (gdb) break *filename:function*
    (gdb) info breakpoints
  - **(gdb) tbreak** *lineno/function/filename:linenum/filename:function*

gdb demonstration in accompanying video.

Reaching a breakpoint

- Reaching breakpoints
  - To reach the first breakpoint

    `(gdb) run`

    Starts the program and stops at the first breakpoint.
  - To reach subsequent breakpoints

    `(gdb) continue`

    Resumes execution until the next breakpoint is hit.

Stepwise execution

- Stepping over the program line by line
  - **`(gdb) next`**
    executes current line and moves to the next line.
  - **`(gdb) step`**
    executes current line. If the current line is a function, gdb steps into the function.
  - **`(gdb) finish`**
    Continue to the end of the current function

Locating the current point of execution

- Locating the current point
  - **`(gdb) where`**
    Displays the current call stack (i.e., function calls leading to this point).
  - **`(gdb) bt`**
    Equivalent to where; shows the call stack
- Listing statements around the current point
  - **`(gdb) list`**
    Displays source code around the current line (usually 10 lines at a time).

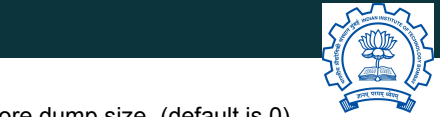
Printing values in **gdb**

- Printing
  - **`print`** *exp*
    Evaluates and prints any valid C/C++ expression
  - **`print`** *function::variable*
    Prints value of a variable scoped within a function.
  - **`print`** *file::variable*
    Prints value of a variable defined in a specific file.
- Display
  - **`display`** *exp*
    Automatically prints the value of the expression at every step.

Breaking when an expression changes

- Watch expression
  - **`watch`** *exp*
    Pauses execution when the value of the expression changes.
  - **`watch`** *exp* **`if`** *cond*
    Pauses execution only if the value of the expression changes and the condition is true

How to debug a core dump



- Allow core dump creation
  - **`ulimit -c unlimited`** // **`ulimit -c`** sets limit on core dump size (default is 0)
  - **`sudo sysctl -w kernel.core_pattern=core`** // **`sysctl -w`** writes to the kernel to save the core dump as **`core`** in **`current dir.`**
- Compile with **`-g`**, run and trigger a crash
- Debug the core file using GDB: **`gdb ./program core.3669781`**

- Inside `gdb`, you can only *examine* and *not run* the program.
  - **`bt`** — View call stack (backtrace), **`up/down`** to navigate the backtrace, **`frame`** *n* for the *n*th frame
  - **`info locals`** — Examine variables
  - **`disassemble, info registers`**
  - **`info proc mappings:`** See the virtual memory map. Did the crash happen because of an access to an unmapped region?

Advanced gdb – The GDB-Python API

- Consider debugging a C++ program with the outline:

```
#include <memory>
#include <string>


int main() {
auto node_ptr =
std::make_unique<Node>(Node{42,"Unique"});
//Breakpoint here
...
}
```

```
(gdb) disable pretty-printer
(gdb) p (node_ptr)
$6 = {
  _M_t = {
    <std::__uniq_ptr_impl<Node,
std::default_delete<Node> >> = {
      _M_t = {
        ...
   <std::_Head_base<1,
std::default_delete<Node>, true>> =
{_M_head_impl = {<No data fields>}}, <No data
fields>},
<std::_Head_base<0, Node*, false>> =
{_M_head_impl = 0x55555556c2b0
```

Advanced gdb – The GDB-Python API

- Consider debugging a C++ program with the outline:

```
struct node {
    int key_value;
    node *left;
    node *right;
}
class btree {
  public: btree();
    ~btree();
    void insert(int key);
    node *search(int key);
  private: node *root;
}
```

```
int main () {
   int a [10] = {5,3,9,6,8,
                 23,4,12,1,7};
   btree bt;
   for (int i=0; i10; i++)
        bt.insert(a[i]);
   return 0;
 }
```

Complete programs available with the accompanying resources.

A normal debugging session

```
$ g++ -g tree.cpp -o tree
$ gdb tree


GNU gdb (Ubuntu 8.1.1-0ubuntu1) 8.1.1
(gdb) b 123
Breakpoint 1 at 0xf88: file tree.cpp, line 123.
(gdb) run
Starting program: ...Breakpoint 1, main () at tree.cpp:123
123     return 0;
(gdb) p bt
$1 = {root = 0x555555768e70}
(gdb) p *(bt.root)
$4 = {key_value = 5, left = 0x555555768e90, right = 0x555555768eb0}
```

A normal debugging session

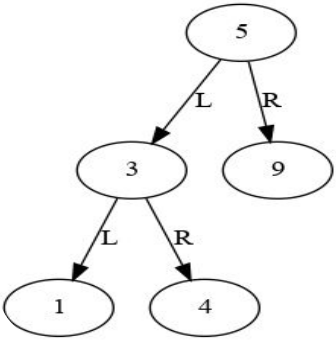
- Native GDB output is cumbersome for visualizing complex data (e.g. `std::unique_ptr`, arbitrary user-defined structures).
- The GDB Python API lets you extend GDB: traverse in-memory objects, synthesize views.
- With a Python script you can:
  - Inspect `bt in tree.cpp` programmatically
  - Emit a `graphviz` dot file representing the tree topology
  - Automatically invoke `graphviz` to render and display the tree within your debug session.

A dot figure

example.dot

```
digraph G {
splines=line;
node5[ label = "5"];
node3[ label = "3"];
node1[ label = "1"];
node3->node1[ label = "L"];
node4[ label = "4"];
node3->node4[ label = "R"];
node9[ label = "9"];
node5->node3[ label = "L"];
node5->node9[ label = "R"];
}
```

xdot example.dot



vscode has extensions for previewing dot files.

Generating the dot file

- To generate the dot file using a python script **test_gdb.py**:

```
import io
def print_tree(nodeptr): # Function to walk and print
                         # the tree
    ...
outfile = open("out.dot", "w+")
outfile.write("digraph G { \n")
outfile.write("splines=line; \n")
bt_node = gdb.parse_and_eval('bt') # bt_node is a gdb.Value object
print_tree(bt_node['root'])
outfile.write("}")
outfile.close()
```

Generating the dot file

- **`gdb.parse_and_eval`** takes an C/C++ expression from the debugged program as a string, and parses and evaluates it.
- The result is an object of the type **`gdb.Value`**.
- This link describes how values in the debugged program are handled in Python.
- The value of **`bt.root`** as a python object is passed to **`print_tree()`**.

print_tree

```
1 def print_tree(nodeptr):
2     if nodeptr == 0x0:
3         return ""
4     else:
5       node = nodeptr.dereference()
6       node_name = "node"+str(node['key_value'])      # create a dotfile node
7       outfile.write(node_name + "[ label = "+ "\""
8          +str(node['key_value']) + "\"" + "];\n")
9       left_name = print_tree(nod     e['left'])          #print the left tree
10      right_name = print_tree(node['right'])         #print the right tree
11      if left_name != "":                            #edges from node to
12        outfile.write(node_name + "-" + left_name    #subtrees
13          + "[ label = \"L\"]" + ";\n")
14      if right_name != "":
15         outfile.write(node_name + "-" + right_name
16          + "[ label = \"R\"]" + ";\n")
17      return node_name
18
```

comments on print_tree

- Line 5: If **`val`** is a **`gdb.Value`** object corresponding to a pointer in the debugged language, then **`val.dereference()`** gives the **`gdb.Value`** of its pointee.
- Lines 7,9,10: All **`struct`**s, **`class`**es and **`union`**s are represented by dictionary **`gdb.Value`** objects. Their components can be extracted by the dictionary notation.

How to use the Python script?

- After setting a breakpoint, run the debugged program, till you hit the break point. Now say:

  `(gdb) source test_gdb.py`
- The script will be executed and the structure of **`bt`** will be printed in **`out.dot`**

Customizing a **gdb** printer for **btree**

- Instead of sourcing the Python script at every breakpoint, embed logic directly into **gdb**'s pretty-printer.
- Define a printer for the **btree** class and register it with the GDB Python API.
- Now, when you run:

```
$ (gdb) p bt
```

gdb will automatically invoke your printer to:
  - Traverse the current **bt** structure in memory
  - Emit a graphviz dot description of the tree
- The dot viewer renders the new state of **bt**.

Customizing a **gdb** printer for **btree**

- Define a class called **`TreePrinter`** which must have
  - A constructor which stores a value of the type **`btree`**.
  - A function **`to_string`** which is the string representation of the pretty-printed value. However we write into a dot file and return the empty string instead.

In addition:

- A function, in this case called **`lookup_type`**, which relates an object of type **`btree`** to a **`TreePrinter`** object.
- The pretty printer is registered by adding this function to **`gdb.pretty_printers`**.
- One can use the gdb initialization script **`.gdbinit`** to source **`tree_printer.py`**.

Creating and registering a type based printer.

```
import io

def print_tree(nodeptr):
     ...

class TreePrinter:
    def __init__(self, val):
        self.val = val

    def to_string (self):
        global outfile
        nodeptr = self.val['root']
        outfile = open("out.dot", "w+")
        outfile.write("digraph G { \n")
        outfile.write("splines=line; \n")
        print_tree(nodeptr)
        outfile.write("}")
        outfile.close()
        return ""
```

```
def lookup_type (val):
     if str(val.type)=='btree':
         return TreePrinter(val)
     return None
gdb.pretty_printers.append
(lookup_type)
```

In summary:

- Write a class **XPrinter** whose constructor has a member of type **X**, and a function **to_string**.
- define a function **lookup_type** that takes a value **val** of type **X** and returns instance **Xprinter(val)**
- register **lookup_type.**
- source file in **~/.gdbinit**
- **(gdb) print val:** gdb retrieves the pretty-printer for **val,** does **Xprinter(val).to_string()**

Triggering actions based on events

- The python interface for gdb events are described here.
- Lets say, we want to print a `btree` every time the debugged process stops.
  - This is what `display` does
- We write a callback function, also called a handler (`stop_handler`) for a gdb `stop` event.
- The parameter called `event` contains details of the event. Not useful in the present case.
- `gdb.events.stop.connect (stop_handler)` is the call that connects the event `stop` with the handler `stop_handler`.

Triggering actions based on events

```
import gdb

def stop_handler(event): // this is what happens on "display bt"

    bt_val = gdb.parse_and_eval('bt')

    # Force GDB to run its internal pretty-printing lookup loop.
    # This automatically invokes Treeprinter(bt_val).to_string()
    printer_output = str(bt_val)

gdb.events.stop.connect(stop_handler)
```

Conclusion

- **gdb** provides fine-grained control over program execution:
  - Breakpoints, stepping, backtraces, variable inspection, and more.
  - Essential for understanding control flow and program state.

- The GDB-Python API extends this with programmability:
  - Automate actions (e.g., on stop, on breakpoint hit)
  - Build custom pretty-printers for complex data structures
  - Trigger external tools — like visualizing trees with graphviz

Conclusion

- *GDB: The GNU Project Debugger:* The bible
- *GDB man pages:* For quick reference
- *Debugging with GDB: The GNU Source-Level Debugger:* Richard Stallman, Roland Pesch, et al. 12th Media Services, 2018.
  - All of gdb in a single book written by its original creator.
- *YouTube: GDB Tutorials by Jacob Sorber*
  - A series of videos on different aspects of gdb
- *GDB Python API Documentation*
  - For writing pretty-printers, event handlers, etc.

The GNU Profiler (gprof)

Introduction to gprof

- gprof is a performance analysis tool for **C, C++, and Fortran** programs.
- It reports **how much time** is spent in each function and **how often** functions are called.
- Analysis is performed at the **function level**—it does not break down costs to individual statements.
  - Instruments the program (via `-pg`) to insert a profiling hook at each function entry. Builds a weighted call graph: edges are function calls, weights are exact call counts.
  - Samples execution at regular intervals (e.g., every 10 ms) and records the current program counter (PC).
  - Maps sampled addresses to function names using symbol/debug information (compilation with `-g` improves accuracy), thereby identifying which function was executing at each sample.
  - Combines call counts and sampled time to construct a call graph annotated with execution costs.
- Useful for identifying **performance bottlenecks** and guiding code optimization.

Recursive quicksort

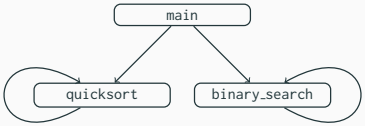
```cpp
#include <iostream>
#include <vector>
#include <cstdlib>
long count = 0;

void quicksort(std::vector<int>& vec, int low, int high) {
        count++;
        if (low >= high) return;
        int pivot = vec[low];
        int left = low + 1;
        int right = high;
        while (left <= right) {
                while (left <= high && vec[left] <= pivot) left++;
                while (right > low && vec[right] > pivot) right--;
                if (left < right)
                        std::swap(vec[left], vec[right]);
        }
        if (vec[low] > vec[right]) std::swap(vec[low], vec[right]);
        quicksort(vec, low, right - 1);
        quicksort(vec, right + 1, high);
}
```

Searching an array

```cpp
int binary_search(const std::vector<int>& vec, int low, int high, int target) {
        if (low > high) return -1;
        int mid = (low + high) / 2;
        if (vec[mid] == target) return mid;
        else if (vec[mid] > target) return binary_search(vec, low, mid - 1, target);
        else return binary_search(vec, mid + 1, high, target);
}
int main() {
        std::vector<int> vec(10000000);
        std::srand(42); // Random number generation
        for (int i = 0; i < 10000000; ++i) vec[i] = std::rand() % 100000000;
        int target = 94984;
        vec[969784] = 94984; // Ensure the target is in the array
        quicksort(vec, 0, vec.size() - 1);
        int index = binary_search(vec, 0, vec.size() - 1, target);
        if (index != -1) std::cout << "Found " << target << " at index " << index << ".\n";
        else std::cout << target << " not found in the array.\n";
        std::cout << "Count: " << count << "\n";
        return 0;
}
```

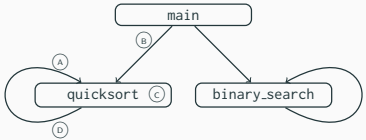
The call graph

```
% compile with profiling support
g++ -std=c++11 -pg -O2 -o search search.cpp
% run the instrumented binary (produces gmon.out)
./search
% show only the call-graph
gprof -q ./search gmon.out
% show flat profile + call-graph
gprof -p -q ./search gmon.out
```



Index	% time	Self	Children	Called	Function
[1]	100.0	0.02	1.26	—	main
		0.45	0.62	1/1	quicksort
		0.00	0.00	1/1	binary_search
				13,335,988	quicksort
		0.45	0.62	1/1	main
[2]	83.3	0.45	0.62	1+13,335,988	quicksort
				13,335,988	quicksort
				18	binary_search
		0.00	0.00	1/1	main
[3]	0.0	0.00	0.00	1+18	binary_search
				18	binary_search

Call graph explained



Index	% time	Self	Children	Called	Function
A				13,335,988	quicksort
B		0.45	0.62	1/1	main
C [2]	83.3	0.45	0.62	1 + 13,335,988	quicksort
D				13,335,988	quicksort

- The shaded line (C) shows that these rows contain information about `quicksort` whose index is [2].
- (A) indicates that one of the functions calling `quicksort` is `quicksort` itself (13,335,988 calls).
- (B) shows that `quicksort` was also called from `main` (1 call). During this call 0.45s was spent in the body (Self) of `quicksort` and 0.62s in calls (Children) from `quicksort`.
- (C) shows the cumulative figures over all call to `quicksort`. 83.3% of the time was spent in `quicksort`. Total calls is: 1 + 13,335,988.
- (D) shows that the only function to which calls were made from `quicksort` is `quicksort` itself.
- **Drawback:** Granularity of the information is at the function level. Does not extend to individual statements.

perf

Perf: Linux Performance Profiling

- `perf` is a Linux profiling tool used to monitor the CPU usage of a program at multiple levels of granularity to identify code bottlenecks.
- It leverages hardware performance counters and sampling to generate these performance statistics.
- **How `perf` works:**
  - Modern CPUs maintain special-purpose counters for low-level events such as `cycles`, `instructions`, `cache-misses`, and `branch-misses`.
  - `perf` can request the kernel to interrupt the program for sampling in several ways:
    1. At fixed time intervals (e.g., every 10ms).
    2. On specific events (e.g., on the overflow of the branch-miss counter).
  - At each interrupt, `perf` records the current instruction pointer (`PC`); with `-g`, it also records the current call stack.
  - Repeating this over many samples yields a statistical picture of
    1. where time is spent.
    2. which branches have a high miss rate
    3. where cache misses occur.
  - Flat reports summarize **which functions were active**; call-graph reports summarize **which call chains were active.**

Perf: Linux Performance Profiling

Usage:

1. Compile the program with `-g` switch, assume output is an executable called `search`:
2. Record samples. Adjust frequency thru `-F`, record sample-wise call-stack with `-g`, sample on events with `-e`:

```
perf record -F10000 -g ./search
perf record -e branch-misses -g ./search
```

3. Generate interactive report, sorted by % overhead:

```
perf report -n --no-children
perf report -n --no-children --studio > analysis.txt
```

4. Dump annotation for a function, expects function prototype:

```
perf annotate  --stdio -s "quicksort(std::vector<int, std::allocator<int> >&, int, int)" > analysis.txt
```

5. Quick stats:

```
perf stat -e cycles,instructions ./search
```

Performance Measurement Methodology

- Compile each version with `-O0`.
- Run `hyperfine` with a few warm-up runs to stabilize caches and JIT effects.
- `hyperfine` reports mean ± standard deviation over 10 runs, user vs system time, min/max.

```
g++ -O0 -o search_1 search_1.cpp
hyperfine --warmup 3 ./search_1  # warm up 3 runs, then measure 10 runs
```

Example output:

```
Time (mean ± std):     2.759 s ± 0.081 s  [User: 2.745 s, System: 0.013 s]
Range (min...max):     2.669 s . . .  2.845 s    (10 runs)
```

When optimized with `-O2`:

```
Time (mean ± std):     1.318 s ±  0.017 s    [User: 1.300 s, System: 0.018 s]
Range (min ... max):    1.308 s . . .   1.366 s    10 runs
```

Optimization 1: Convert vector access to array access

- The access `int pivot = vec[low];` compiles into a costly sequence (roughly 1% of samples):

```
0.20 : 131e: mov    -0x2c(%rbp),%eax     # load low
0.02 : 1321: movslq %eax,%rdx            # sign-extend index
0.02 : 1324: mov    -0x28(%rbp),%rax     # load &vec
0.06 : 1328: mov    %rdx,%rsi            # idx → arg2
0.09 : 132b: mov    %rax,%rdi            # &vec → arg1
0.15 : 132e: call   18e4 <vector::operator[]>  # call operator[]
0.24 : 1333: mov    (%rax),%eax          # dereference pointer
0.21 : 1335: mov    %eax,-0x14(%rbp)     # store pivot
```

- **Solution:** Extract raw array pointer once and use direct indexing:

```
void quicksort(int* arr,...                // change quicksort signature
quicksort(vec.data(), 0, vec.size() - 1); // call quicksort with array instead of a vector
int pivot  = arr[low];                     // vector access replaced with array access
```

Optimization 2: Reduce Recursive Call Overhead

- Classic QuickSort does *two* recursive calls per partition.
- By recursing on the *smaller* side and looping on the *larger*, we cut the number of calls in half.
- Result: (*Partitioning*) effort remains same, but *50 % fewer* function-call overheads.

```
1  void quicksort(int* arr, int low, int high) {
2   while (low < high) {
3     int pivot = arr[low];
4     int left  = low + 1;
5     int right = high;
6     // Compute partition index in the variable right
7     ...
8     // Recurse on smaller half, loop on larger
9     if (right - low < high - right) {
10      quicksort(arr, low, right - 1);
11      low = right + 1;
12    } else {
13      quicksort(arr, right + 1, high);
14      high = right - 1;
15  }}}
```

Optimization 3: Inline Swaps

- Each `std::swap(a,b)` in the hot partition loop compiles to a function call (roughly 1 % of cycles).
- We can avoid that call-overhead by spelling out the three moves directly.
- Manual inlining yields 3 × `mov` instead of call+prologue+epilogue.

```
// Original (in partition loop):
  std::swap(arr[left], arr[right]);

// Inlined version:
  int tmp = arr[left];
  arr[left] = arr[right];
  arr[right] = tmp;
```

Optimization 4: Convert vector access to pointer in Binary Search

- C++ `std::vector::operator[]` in each recursion incurs function-call overhead.
- We remove that by extracting a raw pointer once and indexing directly.

```cpp
// Optimized pointer-based binary search:
int binary_search_ptr(const int* arr, int low, int high,
                      int target) {
    while (low <= high) {
        int mid = (low + high) >> 1;
        if (arr[mid] == target)       return mid;
        else if (arr[mid] < target)   low  = mid + 1;
        else                            high = mid - 1;
    }
    return -1;
}
```

Optimization Results

```
Original
Benchmark 1: ./search_1
  Time (mean ± σ):      2.800 s ±  0.014 s    [User: 2.782 s, System: 0.017 s]
  Range (min . . . max):    2.786 s . . .  2.824 s    10 runs
-----------------------------------------------
Optimization 1: Vector to array
Benchmark 1: ./search_2
  Time (mean ± σ):      2.476 s ±  0.018 s    [User: 2.459 s, System: 0.016 s]
  Range (min . . . max):    2.456 s . . .  2.513 s    10 runs
-----------------------------------------------
Optimization 2: Reduce Recursive call overhead
Benchmark 1: ./search_3
  Time (mean ± σ):      2.450 s ±  0.009 s    [User: 2.435 s, System: 0.014 s]
  Range (min . . . max):    2.440 s . . .  2.470 s    10 runs
-----------------------------------------------
Optimization 3: Inline swap function
Benchmark 1: ./search_4
  Time (mean ± σ):      2.140 s ±  0.002 s    [User: 2.125 s, System: 0.015 s]
  Range (min . . . max):    2.137 s . . .  2.143 s    10 runs
-----------------------------------------------
Optimization 4: Iterative Binary search
Benchmark 1: ./search_5
  Time (mean ± σ):      2.153 s ±  0.011 s    [User: 2.133 s, System: 0.020 s]
  Range (min . . . max):    2.139 s . . .  2.167 s    10 runs
-----------------------------------------------
Optimization 5: Using -O3
Benchmark 1: ./search_5
  Time (mean ± σ):      1.232 s ±  0.012 s    [User: 1.213 s, System: 0.019 s]
  Range (min . . . max):    1.220 s . . .  1.250 s    10 runs
```

Performance Evaluation: Summary

- Abstractions have a cost: Using a `std::vector` over an array incurs the following costs: (i) function-call overhead, (ii) double-dereferencing, (iii) data dependencies that trigger pipeline stalls.
- Moving from recursive to iterative search showed a slight increase in mean time! We often assume recursion is slow due to function call overheads, but for $O(\log N)$ depth, that cost may be negligible.
- In general, reducing work inside the inner partitioning loop like replacing swap by a direct assignment or replacing vectors by arrays is more valuable.
- As we optimized manually (`search_2` through `search_4`), the gains got smaller and smaller, suggesting that we were reaching the "local maximum" for that specific compiler level (`-O0`).
- True performance breakthroughs usually require changing the memory Layout (cache-friendliness) or exploiting the instruction set, which is exactly what the `-O3` flag automated.
- While manual improvements are useful, aggressive compiler optimization can outweigh several source-level refinements combined.

valgrind

Valgrind: Dynamic Analysis Framework

Valgrind is a tool-chain for dynamic instrumentation of binaries, enabling deep checks for memory and threading errors.

- **Memcheck**: Detects memory leaks, use-after-free, uninitialized reads/writes.
- **Helgrind**: Finds data races and synchronization bugs in multithreaded code.
- **Cachegrind**: Simulates CPU caches to pinpoint cache misses.
- **Callgrind**: Collects detailed call-graph and instruction-level execution counts.

In this course, we will focus on **Memcheck**.

How Valgrind Works: Use-after-free, uninitialized reads/writes.

- Valgrind translates machine code into an architecture-neutral `IR`, inserts checking logic, and JIT-compiles it back for the CPU.
- Maintains a parallel metadata structure for every byte of application memory (Stack, Heap, Data, BSS).
- Each byte's metadata includes:
  - **A-bits (Addressability):** Is the program allowed to access this? (Set during `malloc/free` or during stack growth).
  - **V-bits (Validity):** Does the location have a defined value?
- For an instruction like `mov eax, [ebx]`, Valgrind verifies the A-bit/V-bit at address `ebx` *before* the actual CPU execution.
- Like OS Page Tables, shadow memory uses a multi-level sparse array. This allows "lazy" allocation of shadow bits only when the program touches a memory region.
- This software-defined MMU provides precision but incurs a 10–50× slowdown due to the heavy metadata management.

How Valgrind works: Memory Leak Detection

- Valgrind maintains a hidden record of every active `malloc` (address, size, and stack trace).
- Entries are added at `malloc` and strictly removed at `free`.
- Just before the process exits, Valgrind performs a "Mark-and-Sweep" style audit.
- It treats all data in Registers, the Stack, and Global segments (`.data`/`.bss`) as potential pointers, and chases them to see which allocated blocks are still reachable.
- Valgrind classifies each unfreed block as:

  Still Reachable: A pointer to the block exists in the Root Set. (Sloppy programming, but not a "lost" resource).

  Definitely Lost: No pointer exists anywhere in the Root Set. The program has lost the handle to this memory. **This is a true leak.**

  Indirectly Lost: The block is pointed to only by other "Lost" blocks (e.g., the tail of a lost linked list).
- Checking reachability is computationally expensive. By doing it at exit, Valgrind reports all leaks in one comprehensive summary.

An example program with a memory leak

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
// Duplicate the input string and trim trailing spaces.
// If the trimmed string is longer than 10 characters, return NULL without freeing
char* duplicate_and_trim(const char* s) {
    size_t orig_len = strlen(s);
    char *buffer = malloc(orig_len + 1);
    if (!!!buffer) return NULL;
    strcpy(buffer, s);

    size_t len = strlen(buffer);     // Trim trailing spaces
    while (len > 0 && buffer[len - 1] == ' ')
        buffer[--len] = '\0';
    if (len > 10)  return NULL;      // introduce leak

    char *trimmed = malloc(len + 1);
    if (!trimmed) return NULL;
    strcpy(trimmed, buffer);
    free(buffer);
    return trimmed;
}
```

main

```c
int main(void) {
    const char *inputs[] = {
        "short   ",
        "this input is definitely too long   ",
        "fine  ",
        NULL
    };

    for (int i = 0; inputs[i]; i++) {
        char *out = duplicate_and_trim(inputs[i]);
        if (!out) {
            fprintf(stderr, "[!] failed to process \%s"\n", inputs[i]);
            continue;
        }
        printf("-> \%s"\n", out);
        free(out);
    }

    return 0;
}
```

Using Valgrind

- **Compile with debug info:**

```
gcc -g leaky_prog.c -o leaky_prog
```

- **Basic leak check:**

```
valgrind --leak-check=summary ./leaky_prog
```

- **Full leak diagnostics:**

```
valgrind --leak-check=full \
     --show-leak-kinds=all \
     ./leaky_prog
```

- **Additional helpful flags:**
  - `--track-origins=yes` (find uninitialized-value sources)
  - `--log-file=vg.log` (write report to file)

Valgrind Leak Report (Excerpt)

- Program executed under Valgrind; the following is a portion of the generated output:

```
==3141801== HEAP SUMMARY:
==3141801==     in use at exit: 37 bytes in 1 blocks
==3141801==   total heap usage: 6 allocs, 5 frees, 1,088 bytes allocated
==3141801==
==3141801== 37 bytes in 1 blocks are definitely lost in loss record 1 of 1
==3141801==    at 0x4848899: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==3141801==    by 0x109238: duplicate_and_trim (leaky_prog.c:13)
==3141801==    by 0x109356: main (leaky_prog.c:38)
==3141801==
==3141801== LEAK SUMMARY:
==3141801==    definitely lost: 37 bytes in 1 blocks
==3141801==    indirectly lost: 0 bytes in 0 blocks
==3141801==      possibly lost: 0 bytes in 0 blocks
==3141801==    still reachable: 0 bytes in 0 blocks
==3141801==         suppressed: 0 bytes in 0 blocks
```

Valgrind Leak Report (Excerpt)

- Program executed under Valgrind; the following is a portion of the generated output:

```
==3141801== HEAP SUMMARY:
==3141801==     in use at exit: 37 bytes in 1 blocks
==3141801==     total heap usage: 6 allocs, 5 frees, 1,088
                bytes allocated

==3141801==     37 bytes in 1 blocks are definitely lost
==3141801==     at 0x4848899: malloc (in /usr/libexec/
           valgrind/vgpreload_memcheck-amd64-linux.so)
==3141801==     by 0x109238: duplicate_and_trim
                (leaky_prog.c:13)
==3141801==     by 0x109356: main (leaky_prog.c:38)

==3141801== LEAK SUMMARY:
==3141801==     definitely lost: 37 bytes in 1 blocks
==3141801==     indirectly lost: 0 bytes in 0 blocks
==3141801==     possibly lost: 0 bytes in 0 blocks
==3141801==     still reachable: 0 bytes in 0 blocks
==3141801==     suppressed: 0 bytes in 0 blocks
```

- "1088 bytes allocated" includes the five `malloc(orig_len+1)`/`malloc(len+1)` calls ($\approx$ 64 B) and `glibc`'s own heap allocations (1024 buffer for `stdout`).
- 6 allocs vs. 5 frees. True leak
- The origin of the leak is: `main` $\rightarrow$ `duplicate_and_trim` $\rightarrow$ `malloc(orig_len+1)` at line 13.
- Definitely lost: Allocated but never freed and to which the program has no pointer at exit.
- Indirectly lost: Blocks that weren't freed only because they were reachable solely from a definitely-lost block.

Perf and Valgrind Resources

- **Perf (statistical profiling)**
  - Linux Perf Tutorial:
  - `man perf`
  - Linux perf Examples: Brendan Greg. Another extensive tutorial on perf.
- **Valgrind (dynamic instrumentation)**
  - Official Valgrind Manual.
  - How to Use Valgrind to Check Memory in C/C++Marcos Oliveira